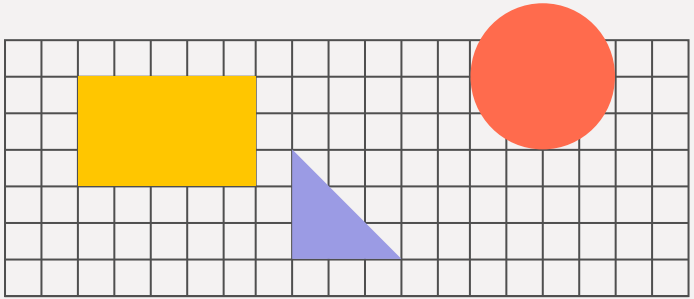
SOCKETS

Sistemas Operativos Curso 2025-26



LA SALLE
RAMON LLULL UNIVERSITY

¿Qué file descriptors conocemos hasta ahora?

- ❏ I/O locales (pantalla, teclado, etc)
- ❏ Ficheros (.txt, .dat, etc)
- ❏ Pipes

Sockets!

¿Cómo conectamos procesos?

En la misma máquina?

En diferentes máquinas?

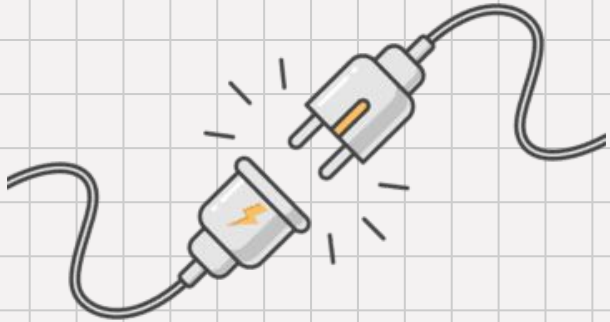
En diferentes continentes?

La solución es con SOCKETS!

¿Que son los Sockets?

Herramienta de comunicación entre procesos

- ➔ Comunicaciones locales y remotas
- ➔ Configuradas con IP y Port
- ➔ Son un file descriptor



Tipos de Sockets

TCP

Orientados a conexión

- Se establece una conexión entre dos puntos
- Los datagramas se empaquetan automáticamente y se reenvían si se pierden.

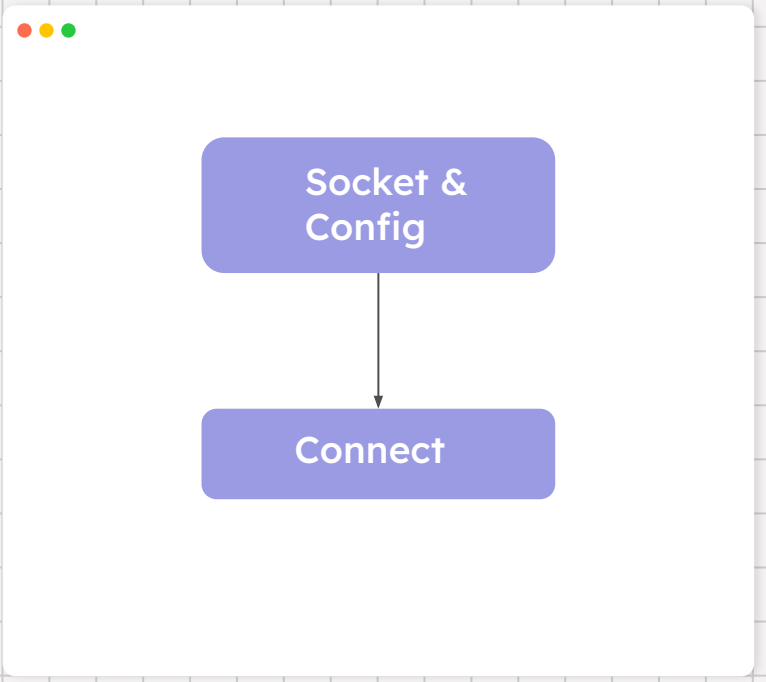
UDP

NO orientados a conexión

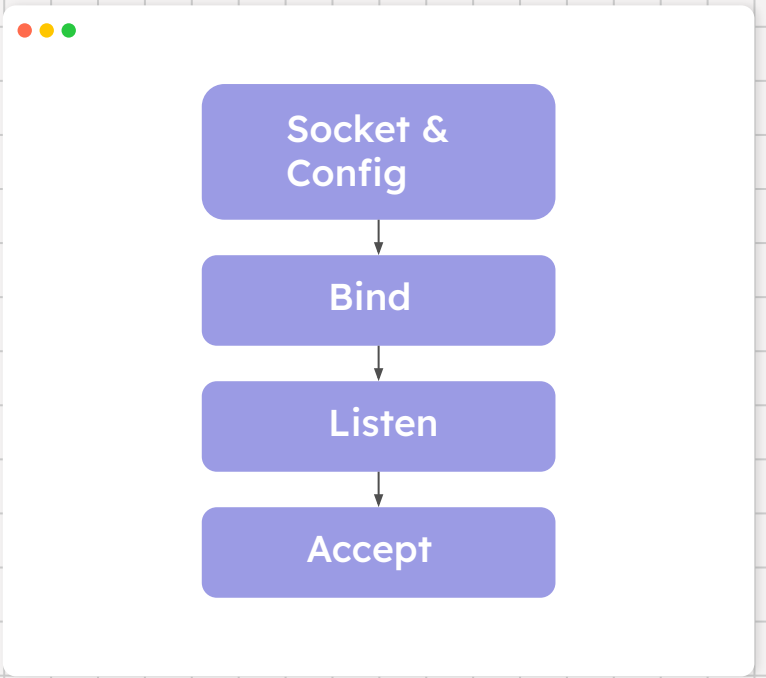
- No se establece una conexión
- No se garantiza que la información se reciba
- Es más rápido

¿Cómo se crean los Sockets?

Cliente



Servidor



Struct: sockaddr

```
#include <sys/socket.h>
```

```
struct sockaddr_in {
    uint8_t sin_len; //Mida del objecte. No utilitzar.
    sa_family_t sin_family;//Família del socket
    in_port_t sin_port; //Número del port en Big Endian
    struct in_addr sin_addr; //Struct con entero indicando @IP
    char sin_zero[8]; //'\0'
};
```

Esta estructura nos ayuda a configurar parámetros del socket como IP, port, etc.

No se usa **sin_len** ni **sin_zero**

Función: socket

```
#include <sys/socket.h>
```

Retorna el file descriptor asociado al socket (todavía falta la conexión!)

```
int socket(int family, int type, int addrlen);
```

int family: AF_INET, AF_INET6, AF_BLUETOOTH, etc.
int type: especifica el tipo de conexión.

- ❏ SOCK_STREAM (orientado a conexión)
- ❏ SOCK_DGRAM (NO orientado a conexión)

int protocol: define el protocolo a usar.

- ❏ 0 por defecto, determinado por el SO.
- ❏ IPPROTO_TCP, IPPROTO_UDP, etc.

Función: connect

```
#include <sys/socket.h>
```

Ejecuta el Three-Way-Handshake si el destino lo permite.

```
int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

int sockfd: el file descriptor del socket.
struct servaddr: configuración con la información del destino.
socklen_t addrlen: tamaño de la estructura servaddr.

Retorna -1 si hay error, 0 otro caso.

Funciones útiles

```
#include <arpa/inet.h>
```

Funciones para convertir ports de Big Endian/formato del host.

```
uint16_t htons(uint16_t port);
```

```
uint16_t ntohs(uint16_t hostshort);
```

Funciones para convertir IP a binario (IPv4 + IPv6)

```
int inet_pton(int af, const char *src, void *dst);
```

```
int inet_aton(const char *cp, struct in_addr *inp);
```

Función: bind

```
#include <sys/socket.h>
```

Le indica al S.O. que la aplicación espera información de una IP y port.

```
int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
```

int sockfd: el file descriptor del socket.
struct myaddr: es necesario hacer cast a (void*).

- ❏ Si se espera una conexión local, se utiliza INADDR_ANY como IP.
- ❏ Si se espera una conexión remota, se utiliza inet_pton.

socklen_t addrlen: tamaño de la estructura myaddr.

Retorna -1 si hay error, 0 otro caso.

Función: listen

```
#include <sys/socket.h>
```

Abre el puerto asignado al socket y determina el número máximo de conexiones.

```
int listen(int sockfd, int backlog);
```

int sockfd: el file descriptor del socket.
int backlog: número máximo de conexiones a aceptar.
Retorna -1 si hay error, 0 otro caso.

Función: accept

```
#include <sys/socket.h>
```

Crea un nuevo file descriptor con una conexión activa. Es una función que bloquea el sistema.

```
int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

int sockfd: el file descriptor del socket.
struct cliaddr: indica la dirección del que ha solicitado la conexión (void*).
Socklen_t addrlen: tamaño de la estructura cliaddr.

Función: close

```
#include <unistd.h>
```

Si el socket no está activo, no se aceptan más conexiones. Si tiene una conexión activa, se cierra la conexión.

```
int close(int sockfd);
```

int sockfd: el file descriptor del socket.

Funciones write/read

```
#include <unistd.h>
```

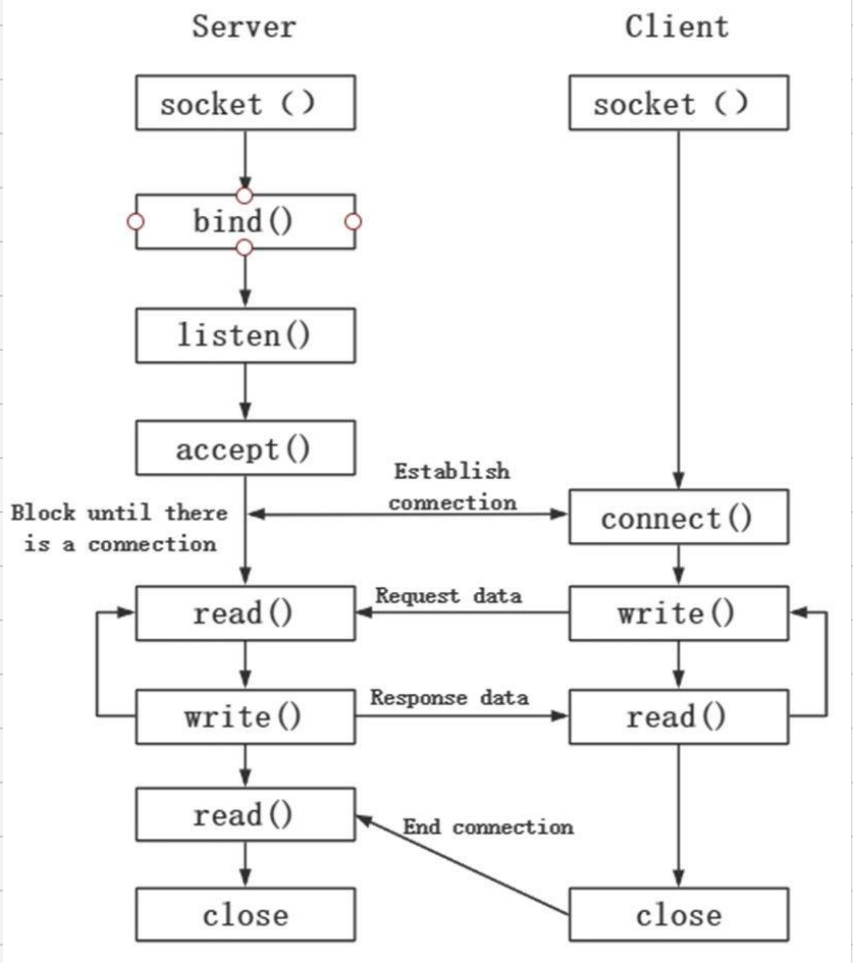
write: bloquea el proceso hasta que se haya escrito todo al buffer del S.O.

Read: bloquea el proceso hasta que aparezcan datos en el buffer del S.O.

- ❏ Es necesario comprobar que se hayan leído todos los datos.
- ❏ 0 bytes leídos = se ha cerrado la conexión.

Tamaño variable: se usa ssize_t = signed size_t = signed integer
Tamaño fijo: se puede utilizar uint32_t, uint16_t, uint8_t, etc.

Ejemplo de flujo



PREGUNTAS?

DEMO

Preguntas

- ❑ Qué sucede si no se han leído todos los bytes esperados?
- ❑ Cómo hacemos para que el servidor espere un mensaje del cliente?
- ❑ Cómo comprobamos que el cliente haya cerrado la conexión?
- ❑ Que pasa si el port está ocupado por otro proceso?
- ❑ Que pasa si dos clientes envían mensajes a la vez?

Preguntas / Respuestas

- Qué sucede si no se han leído todos los bytes esperados?
  read() NO garantiza que todos los bytes esperados se hayan leído. Solución?
  Leer el resto (totalBytes - bytesRead)
- Cómo hacemos para que el servidor espere un mensaje del cliente?
  El servidor hace read() y el cliente write()
- Cómo comprobamos que el cliente haya cerrado la conexión?
  numBytes = read() → numBytes == 0 → close()
- Que pasa si el port está ocupado por otro proceso?
  Error del bind() → probar otro IP/port o informar al usuario.
- Que pasa si dos clientes envían mensajes a la vez?
  Si solo hay un thread → el segundo cliente tiene que esperar al servidor
  Si hay más threads → se gestionan independientemente (en paralelo)